CUDA and GPU Research Track

Getting Started Guide for Projects 1 to 10

#	Project	Duration	Focus
1	Custom CUDA Kernel Library for Common ML Operators	10 weeks	Hand-tuned matmul, layernorm and softmax for your production model shapes
2	GPU Memory Profiling and Optimization Toolkit	7 weeks	Detecting uncoalesced access, bank conflicts and other memory bottlenecks automatically
3	Multi-Stream CUDA Pipeline for Overlapping Compute and Data Transfer	6 weeks	Hiding host-device copy time behind kernel execution
4	Warp-Level Primitives for Fast Reduction Operations	5 weeks	Warp-shuffle sum, max and argmax versus shared-memory reductions
5	Triton-Based Kernel Auto-Tuning Framework	8 weeks	Searching launch and tiling configurations per GPU, faster than manual grid search
6	Custom CUDA Graph Capture for Reduced Kernel-Launch Overhead	6 weeks	Capturing and replaying a fixed kernel sequence to remove CPU launch cost
7	Multi-GPU Collective Communication Benchmark Suite (NCCL)	7 weeks	Benchmarking all-reduce, all-gather and friends across interconnect topologies
8	GPU Kernel Fusion Compiler Pass for a Small DL Model	9 weeks	Fusing elementwise operations into single kernels with torch.fx and Triton
9	Mixed-Precision CUDA Training Kernel Benchmarks	8 weeks	Throughput versus convergence trade-offs of FP32, TF32, FP16, BF16 and FP8
10	Custom CUDA Kernel for Sparse Matrix Operations in ML Pipelines	9 weeks	Sparse matmul tuned to your model's sparsity pattern, compared with cuSPARSE and dense

How to use this guide

- Each project has: overview, research question, architecture, prerequisites and hardware, what to build, baselines, experiments, metrics, repo layout, week-by-week plan matched to the track duration, definition of done, final deliverables and engineering rules.
- Projects are numbered 1 to 10 in the order of the track sheet; durations come from the sheet.
- All projects need an NVIDIA GPU. Hardware-counter profiling (Nsight Compute) is often blocked on shared notebook platforms, so prefer a local or rented GPU where you have admin rights. Project 7 needs 2+ GPUs; project 9 FP8 needs Hopper or Ada.
- Tool versions and metric names change; record the GPU, driver, CUDA, PyTorch and Triton versions with every result.

Common setup (all projects)

```
nvidia-smi && nvcc --version && ncu --version && nsys --version
python -m venv .venv && source .venv/bin/activate
# install the CUDA build of PyTorch from pytorch.org, then:
pip install triton numpy pandas matplotlib pytest
mkdir -p src bench tests results docs scripts
```

Benchmarking method (use in every project)

- Write correctness tests against a PyTorch or CPU reference BEFORE measuring speed; use compute-sanitizer (memcheck, racecheck) on kernels.
- Warm up, then time with CUDA events (not wall-clock without synchronization); report the median and spread of 100+ iterations.
- Keep GPU, clocks, dtype and shapes identical when comparing; watch for thermal throttling and other processes on the GPU.
- Compare against vendor baselines (cuBLAS, cuSPARSE, CUB, NCCL) and report percent of hardware peak, not only speedup.
- Use Nsight Systems for timelines and Nsight Compute for kernel metrics; keep only small report files in the repo.
- Check every CUDA call for errors (CUDA_CHECK macro) and every kernel launch with cudaGetLastError.

Event-timing pattern

```
cudaEvent_t a, b; cudaEventCreate(&a); cudaEventCreate(&b);
for (int i = 0; i < WARMUP; ++i) kernel<<<g, blk>>>(args);
cudaEventRecord(a);
for (int i = 0; i < ITERS; ++i) kernel<<<g, blk>>>(args);
cudaEventRecord(b); cudaEventSynchronize(b);
float ms; cudaEventElapsedTime(&ms, a, b);  // per-iter = ms / ITERS
```

Project 1: Custom CUDA Kernel Library for Common ML Operators

Hand-tuned matmul, layernorm and softmax for your production model shapes

Duration	Core technologies
10 weeks	CUDA C++, Nsight Compute, cuBLAS/cuDNN baselines, PyTorch C++ extension

1. Overview

Vendor kernels are tuned for general shapes. If your models use specific hidden sizes, sequence lengths and batch sizes, hand-tuned or fused kernels can win on those shapes. This project builds matmul, layernorm and softmax kernels step by step, profiles each optimization, and benchmarks against cuBLAS and PyTorch.

Research question

On which target shapes and dtypes can hand-written kernels beat cuBLAS / PyTorch, and which optimizations produce the gain?

Expected outcome

A kernel library that beats vendor baselines on target shapes by a measurable margin.

Prerequisites and hardware

Skills: C/C++, basic CUDA (threads, blocks, shared memory), PyTorch basics.

Hardware: Any NVIDIA GPU for development. Tensor Core work needs compute capability 7.0+ (BF16 needs 8.0+). Hardware-counter profiling usually needs a local or rented GPU, not a shared notebook.

2. Architecture / pipeline

Target-shape list -> reference implementations -> naive kernels -> optimized kernels (one optimization at a time) -> correctness tests -> benchmark harness -> Nsight Compute analysis -> PyTorch extension -> final shape-by-shape comparison table.

3. What to build

- Shape list: extract the real (M, N, K), hidden size, sequence length and batch size values from your production models.
- Matmul ladder: naive -> shared-memory tiling -> register blocking -> vectorized loads (float4) -> double buffering -> Tensor Cores (WMMA or mma.sync). Record speedup after each step.
- Layernorm: one block or warp per row, Welford or two-pass variance, warp-shuffle reductions, vectorized access, optional fusion with residual add or bias.
- Softmax: numerically stable (subtract row max), online softmax, warp-per-row for short rows and block-per-row for long rows, optional fused scaling or masking.
- Correctness suite: compare to PyTorch float64 reference with explicit tolerances per dtype (FP32, FP16, BF16).
- Benchmark harness: warmup, CUDA events, median over many iterations, TFLOPS and GB/s computation, CSV output.
- PyTorch extension (torch.utils.cpp_extension) so kernels can be called from Python and dropped into a model.

4. Baselines to compare

Naive CUDA kernel | PyTorch eager | cuBLAS / cuDNN | torch.compile (Inductor) | Your best kernel per shape

5. Experiments to run

Tile and block size sweep | Shape sweep (target shapes plus odd shapes) | dtype comparison (FP32 / FP16 / BF16) | Occupancy vs register usage | Vectorized vs scalar loads | Fusion vs separate kernels | End-to-end model latency

with your kernels swapped in

6. Evaluation metrics

Latency (us) | TFLOPS and percent of peak (matmul) | GB/s and percent of peak bandwidth (layernorm, softmax) | Speedup vs cuBLAS / PyTorch | Max abs and relative error | Nsight Compute: achieved occupancy, memory throughput, warp stall reasons

7. Suggested repository structure

```
cuda_kernels/
  src/matmul/  src/layernorm/  src/softmax/
  python/        extension + bindings
  bench/         harness and CSV results
  tests/         correctness vs reference
  profiles/      .ncu-rep files (small)
  docs/ scripts/ CMakeLists.txt
```

8. Week-by-week plan (10 weeks)

Week	Deliverable
1	Environment, target-shape list, reference implementations, benchmark harness, vendor baselines.
2	Naive matmul, layernorm, softmax kernels with correctness tests.
3	Matmul: shared-memory tiling and register blocking.
4	Matmul: vectorized loads, double buffering, first Nsight Compute analysis.
5	Matmul: Tensor Core version; compare to cuBLAS on target shapes.
6	Layernorm: warp-shuffle and vectorized kernels, residual fusion.
7	Softmax: online and fused variants.
8	PyTorch extension and integration into a real model.
9	Full benchmark sweep, ablations, roofline analysis.
10	Freeze, final report, demo.

9. Definition of done

- All kernels pass correctness tests for each dtype
- Shape-by-shape table vs cuBLAS / PyTorch
- At least one operator beats the vendor baseline on a target shape, or a documented explanation of why not
- Nsight Compute evidence for each major optimization
- Python bindings usable in a model

10. Final deliverables

- Kernel library with build system
- Benchmark harness and raw CSV results
- Profiling reports and roofline plots
- Technical report and demo notebook

11. Critical engineering rules

- Correctness first; never report speed for a kernel that fails tests.
- Beating cuBLAS on large square matmul is very hard; focus on your real shapes, fused ops and small or skinny shapes, and report honestly where you lose.
- Always compare on the same GPU, same dtype and same clocks.
- Report percent of hardware peak, not only speedup.

Project 2: GPU Memory Profiling and Optimization Toolkit

Detecting uncoalesced access, bank conflicts and other memory bottlenecks automatically

Duration	Core technologies
7 weeks	CUDA, Nsight Compute (ncu), Nsight Systems (nsys), Python tooling

1. Overview

Memory-access mistakes silently cost large factors of performance. This project builds a Python toolkit that runs Nsight Compute on a kernel, parses its counters, applies rules to detect inefficiencies and prints concrete fix suggestions, validated on micro-kernels with known bottlenecks.

Research question

Can counter-based rules reliably detect memory-access problems and point to fixes that actually improve runtime?

Expected outcome

A toolkit that identifies and helps resolve at least 3 classes of memory-access bottlenecks.

Prerequisites and hardware

Skills: CUDA basics, memory hierarchy, Python (pandas), command-line profiling.

Hardware: NVIDIA GPU with ncu access. Counter access is often blocked on shared platforms (permission error); use a local or rented machine where you can profile.

2. Architecture / pipeline

Target kernel/binary -> ncu profiling with chosen metric sets -> CSV/JSON parser -> rule engine (thresholds) -> bottleneck classification -> fix suggestions -> HTML/markdown report -> verification against fixed kernel.

3. What to build

- Micro-benchmark suite: for each bottleneck class write a BAD kernel and a FIXED kernel (these are your ground truth).
- Class 1, uncoalesced global access: strided or transposed reads; detect via sectors-per-request and global load efficiency.
- Class 2, shared-memory bank conflicts: column access in a tile; detect via bank-conflict counters; fix with padding or swizzle.
- Class 3, register spilling / local-memory traffic: detect via local load/store counters; fix with launch bounds or restructuring.
- Optional classes: low occupancy, poor L2 hit rate, misaligned access, memory-bound vs compute-bound classification.
- ncu wrapper in Python (subprocess with --csv and --metrics or --set), parser, rule engine with configurable thresholds.
- Report generator: kernel name, detected issue, evidence (metric values), suggested fix, expected benefit.

4. Baselines to compare

Manual reading of Nsight Compute GUI | Toolkit on bad kernels | Toolkit on fixed kernels (should stay quiet) | Toolkit on real kernels (e.g. a matmul or transpose)

5. Experiments to run

Detection on each class with varying stride and tile size | Threshold tuning vs false alarms | Speedup after applying suggested fix | Different GPUs or architectures (metric availability) | Profiling overhead and replay passes | Real-kernel case study

6. Evaluation metrics

Detection precision and recall on micro-kernels | False-positive rate on fixed kernels | Speedup after fix | Profiling time per kernel | Memory throughput percent of peak

7. Suggested repository structure

```
gpu_memprof/
  kernels/       bad and fixed micro-kernels
  profiler/      ncu runner + parsers
  rules/         detection rules + thresholds
  report/        templates
  tests/ docs/ results/
```

8. Week-by-week plan (7 weeks)

Week	Deliverable
1	Set up ncu/nsys, read memory-hierarchy docs, list candidate metrics (check names with ncu --query-metrics).
2	Bad and fixed micro-kernels for 3 classes.
3	ncu wrapper and parser.
4	Rule engine and detection for 3 classes.
5	Fix suggestions, validation of speedups.
6	Real-kernel case study, threshold tuning.
7	Report generator, documentation, demo.

9. Definition of done

- 3 or more bottleneck classes detected
- Precision/recall table on micro-kernels
- Measured speedup from suggested fixes
- Case study on a real kernel
- Usable CLI

10. Final deliverables

- Profiling toolkit (CLI + library)
- Micro-benchmark suite
- Detection evaluation report
- Case-study write-up and demo

11. Critical engineering rules

- Metric names differ across ncu versions and GPUs; verify with ncu --query-metrics and record versions.
- Counter values depend on replay and clocks; repeat measurements.
- A rule is only valid if the fix actually improves runtime; measure it.
- Report profiling overhead honestly.

Project 3: Multi-Stream CUDA Pipeline for Overlapping Compute and Data Transfer

Hiding host-device copy time behind kernel execution

Duration	Core technologies
6 weeks	CUDA streams, cudaMemcpyAsync, pinned memory, CUDA events, Nsight Systems

1. Overview

Copying data to the GPU, computing, then copying back leaves the GPU idle during transfers. By chunking the work and using multiple streams with pinned host memory, copies and kernels can overlap. This project builds the pipeline and quantifies when overlap helps.

Research question

How much wall-clock time can overlap recover as a function of chunk size, stream count and compute-to-transfer ratio?

Expected outcome

A pipeline with measurable wall-clock speedup versus a synchronous baseline.

Prerequisites and hardware

Skills: Basic CUDA, understanding of PCIe transfers.

Hardware: Any NVIDIA GPU. Bidirectional overlap needs a GPU with two copy engines; check with cudaGetDeviceProperties (asyncEngineCount).

2. Architecture / pipeline

Host data (pinned) -> split into chunks -> per-stream: H2D copy, kernel, D2H copy -> events for dependencies -> timeline verified in Nsight Systems -> compare to synchronous version.

3. What to build

- Synchronous baseline: pageable memory, copy all, compute, copy back.
- Pinned-memory version (cudaMallocHost or cudaHostAlloc) with a single stream.
- Chunked multi-stream pipeline with N streams and configurable chunk size.
- Double-buffering variant and event-based dependency control.
- Tunable compute kernel (adjustable arithmetic intensity) to sweep the compute-to-transfer ratio.
- Optional: unified memory with prefetch, and a realistic workload (image batches or matrix tiles).
- Nsight Systems timeline captures proving the overlap.

4. Baselines to compare

Synchronous pageable-memory pipeline | Pinned memory, single stream | Multi-stream pipeline | Theoretical bound: max(transfer, compute) vs sum

5. Experiments to run

Stream count (1, 2, 4, 8) | Chunk size sweep | Compute intensity sweep | Pageable vs pinned | H2D-only vs bidirectional overlap | Unified memory with prefetch | Total data size scaling

6. Evaluation metrics

Wall-clock time | Speedup vs synchronous | Overlap percent (from timeline) | Achieved PCIe bandwidth | GPU idle percent | Gap to theoretical bound

7. Suggested repository structure

```
cuda_streams/
  src/           baseline + pipeline variants
  bench/         sweeps, CSV
  timelines/     nsys reports
  tests/ scripts/ docs/
```

8. Week-by-week plan (6 weeks)

Week	Deliverable
1	Baseline pipeline, timing harness, measure transfer and kernel times separately.
2	Pinned memory and single-stream version.
3	Multi-stream chunked pipeline with events.
4	Sweeps: streams, chunk size, compute intensity; Nsight Systems timelines.
5	Realistic workload and optional unified-memory variant.
6	Analysis versus theoretical bound, report, demo.

9. Definition of done

- Verified overlap in timeline
- Speedup vs synchronous baseline
- Sweep plots
- Explanation of when overlap does not help

10. Final deliverables

- Pipeline source code
- Benchmark results and plots
- Nsight Systems timeline evidence
- Technical report and demo

11. Critical engineering rules

- Use pinned memory; pageable memory prevents real async overlap.
- Maximum gain is limited by the larger of compute and transfer time; do not claim more.
- Validate results for correctness after chunking.
- Synchronize properly before timing.

Project 4: Warp-Level Primitives for Fast Reduction Operations

Warp-shuffle sum, max and argmax versus shared-memory reductions

Duration	Core technologies
5 weeks	CUDA C++, warp shuffle intrinsics, cooperative groups, CUB/Thrust baselines

1. Overview

Classic reductions use shared memory and synchronization. Warp-level shuffles exchange data directly between registers with no shared memory and no block barriers inside a warp. This project implements sum, max and argmax with shuffles and benchmarks them against shared-memory kernels and library reductions.

Research question

How much latency and bandwidth do warp shuffles save over shared-memory reductions across array sizes and block configurations?

Expected outcome

A reduction kernel with measurable latency improvement over the shared-memory baseline.

Prerequisites and hardware

Skills: Basic CUDA, thread/warp model, shared memory.

Hardware: Any NVIDIA GPU (sm_30+ for shuffles). The __reduce_*_sync intrinsics need compute capability 8.0+.

2. Architecture / pipeline

Input array -> per-thread partial (grid-stride, vectorized loads) -> warp reduce via __shfl_down_sync / __shfl_xor_sync -> warp leaders write to shared memory -> first warp reduces -> block result -> final stage (second kernel or atomics).

3. What to build

- Baseline 1: shared-memory tree reduction (progressively optimized as in the classic CUDA reduction tutorial).
- Baseline 2: cub::DeviceReduce and torch.sum / torch.max / torch.argmax.
- Warp reduce with __shfl_down_sync for sum and max; __shfl_xor_sync butterfly variant.
- Argmax: reduce (value, index) pairs with deterministic tie-breaking.
- Block-level two-stage reduction, grid-stride loops, unrolling and vectorized loads.
- Multi-block finalization: second kernel vs atomics vs cooperative groups grid sync.
- Optional: __reduce_add_sync on sm_80+, and a segmented (per-row) reduction.

4. Baselines to compare

Shared-memory tree reduction | cub::DeviceReduce | torch.sum / max / argmax | Warp-shuffle reduction (yours)

5. Experiments to run

Array size sweep (1e3 to 1e9) | Block size | Elements per thread / unroll factor | Scalar vs vectorized loads | dtype (FP32 / FP16 / INT32) | Atomic vs two-kernel finalization | Small-array latency vs large-array bandwidth

6. Evaluation metrics

Latency (us) | Bandwidth GB/s and percent of peak | Speedup vs shared-memory baseline | Numerical error vs float64 | Determinism across runs

7. Suggested repository structure

```
warp_reduce/
  src/   baseline_smem.cu  warp_shuffle.cu  argmax.cu
  bench/ tests/ results/ docs/
```

8. Week-by-week plan (5 weeks)

Week	Deliverable
1	Baselines (shared-memory, CUB, PyTorch), harness, correctness tests.
2	Warp-shuffle sum and max; block-level reduction.
3	Argmax, vectorization, unrolling, finalization strategies.
4	Sweeps and Nsight Compute analysis.
5	Report, demo.

9. Definition of done

- Sum, max and argmax kernels pass tests
- Measured latency/bandwidth gain vs shared-memory
- Comparison with CUB
- Documented numerical behavior

10. Final deliverables

- Kernel source
- Benchmark data and plots
- Short technical report
- Demo

11. Critical engineering rules

- Floating-point sums depend on order; compare against float64 with tolerance and note nondeterminism of atomics.
- Large arrays are bandwidth-bound; shuffles mostly help latency and small or medium sizes. Report honestly.
- Use full-warp masks correctly (0xffffffff) to avoid undefined behavior.

Project 5: Triton-Based Kernel Auto-Tuning Framework

Searching launch and tiling configurations per GPU, faster than manual grid search

Duration	Core technologies
8 weeks	OpenAI Triton, Python, CUDA, Optuna (optional)

1. Overview

Block sizes, warps and pipeline stages that are best on one GPU or shape are often poor on another. This project builds an auto-tuning harness around Triton kernels with smarter search strategies and a persistent per-GPU config cache.

Research question

How close to the exhaustive-search optimum can smarter search get, and how many fewer evaluations does it need?

Expected outcome

An auto-tuner that finds near-optimal configs faster than manual grid search.

Prerequisites and hardware

Skills: Python, basic GPU concepts, willingness to learn Triton's programming model.

Hardware: NVIDIA GPU supported by Triton. For the cross-GPU experiment, access to two different GPU types (e.g. a laptop GPU and a cloud GPU).

2. Architecture / pipeline

Triton kernel + search space -> benchmark function (triton.testing.do_bench) -> search strategy (grid / random / successive halving / Bayesian) -> best config -> JSON cache keyed by (GPU, shape, dtype) -> lookup at runtime.

3. What to build

- Reference Triton kernels: matmul, softmax, layernorm (follow the official Triton tutorials first).
- Search space definition: BLOCK_M/N/K, num_warps, num_stages, group size; invalid-config pruning (shared memory / register limits).
- Benchmark harness with warmup, repetition and correctness check against PyTorch.
- Search strategies: exhaustive grid (ground truth), random search, successive halving, Bayesian optimization, optional cost-model pruning.
- Config cache: JSON file keyed by GPU name, compute capability, kernel, shape bucket and dtype.
- Baseline comparison against Triton's built-in @triton.autotune.
- Report tooling: tuning curves, evaluations-to-quality plots.

4. Baselines to compare

Default hand-picked config | Exhaustive grid search (ground truth) | Random search | triton.autotune | Your best strategy

5. Experiments to run

Evaluation budget vs best-found performance | Search strategy comparison | Shape generalization (does a config transfer to nearby shapes?) | Cross-GPU transfer of configs | Noise robustness (repeat benchmarks) | Cache hit effect on startup time | Kernel types (matmul vs memory-bound)

6. Evaluation metrics

Time to tune | Number of evaluations | Best config as percent of exhaustive optimum | Speedup vs default config | Cache hit rate | Variance of measurements

7. Suggested repository structure

```
triton_tuner/
  kernels/  search/  cache/
  bench/  experiments/  plots/
  tests/ docs/
```

8. Week-by-week plan (8 weeks)

Week	Deliverable
1	Triton tutorials, reference kernels, correctness tests.
2	Benchmark harness and search-space definition.
3	Exhaustive grid search as ground truth on 2-3 shapes.
4	Random search and successive halving.
5	Bayesian search, config pruning.
6	Persistent cache and runtime lookup.
7	Experiments: budget curves, shape generalization, cross-GPU.
8	Report, demo.

9. Definition of done

- Working tuner with at least 3 strategies
- Exhaustive ground truth for comparison
- Per-GPU cache
- Evaluation-vs-quality curves
- Cross-GPU or shape-transfer result

10. Final deliverables

- Auto-tuning framework
- Config cache format spec
- Experiment data and plots
- Technical report and demo

11. Critical engineering rules

- You need exhaustive-search results to claim near-optimal.
- Benchmarks are noisy; repeat and use medians.
- Config caches are GPU-specific; always key by GPU and driver/Triton version.
- Verify correctness of every candidate config.

Project 6: Custom CUDA Graph Capture for Reduced Kernel-Launch Overhead

Capturing and replaying a fixed kernel sequence to remove CPU launch cost

Duration	Core technologies
6 weeks	CUDA Graphs API, PyTorch CUDA Graphs, torch.compile, Nsight Systems

1. Overview

Many small kernels per step make inference CPU-launch-bound: the GPU waits for the CPU to enqueue work. CUDA Graphs record a kernel sequence once and replay it with a single launch. This project measures the benefit in raw CUDA and in PyTorch.

Research question

For which kernel counts, kernel durations and batch sizes do CUDA Graphs reduce latency, and what are the constraints?

Expected outcome

A measurable reduction in per-inference-step CPU overhead.

Prerequisites and hardware

Skills: CUDA streams, PyTorch basics.

Hardware: Any recent NVIDIA GPU; CUDA 11.x or newer.

2. Architecture / pipeline

Fixed inference step -> warmup on side stream -> stream capture -> instantiate graph -> replay with static input/output buffers -> compare against eager launches -> Nsight Systems timeline.

3. What to build

- Synthetic microbenchmark: N tiny kernels per step, graph vs normal launches (raw CUDA stream capture: cudaStreamBeginCapture, cudaStreamEndCapture, cudaGraphInstantiate, cudaGraphLaunch).
- PyTorch version: torch.cuda.CUDAGraph with static tensors, plus make_graphed_callables.
- Realistic model: small MLP or transformer decode step with batch size 1 to 8.
- Comparison with torch.compile(mode='reduce-overhead').
- Graph updates: change kernel parameters without full re-capture, versus re-capture cost.
- Dynamic shapes handling via a graph per shape bucket.
- Timeline analysis showing GPU idle gaps before and after.

4. Baselines to compare

Eager PyTorch | Raw CUDA launches | CUDA Graph replay | torch.compile reduce-overhead

5. Experiments to run

Kernels per step (10 to 1000) | Kernel duration sweep (launch-bound to compute-bound) | Batch size | Capture time and memory overhead | Graph update vs re-capture | Shape buckets for dynamic inputs | Multiple graphs / streams

6. Evaluation metrics

CPU time per step | End-to-end latency p50 / p99 | GPU idle gaps | Throughput | Capture time | Extra memory

7. Suggested repository structure

```
cuda_graphs/
```

```
raw_cuda/  pytorch/  models/
bench/  timelines/  tests/  docs/
```

8. Week-by-week plan (6 weeks)

Week	Deliverable
1	Launch-overhead microbenchmark in eager mode.
2	Raw CUDA graph capture and replay.
3	PyTorch CUDAGraph with static buffers; correctness.
4	Model-level experiments and torch.compile comparison.
5	Updates, dynamic shapes, timelines.
6	Report, demo.

9. Definition of done

- Graph replay produces identical outputs
- Measured CPU overhead reduction
- Sweep showing when graphs help
- Constraints documented

10. Final deliverables

- Benchmarks and graph wrappers
- Timelines before/after
- Results report and demo

11. Critical engineering rules

- Graphs need static memory addresses and fixed shapes; document how you handle this.
- Always warm up before capture.
- Gains vanish when kernels are long; report that regime too.
- Verify outputs match eager mode.

Project 7: Multi-GPU Collective Communication Benchmark Suite (NCCL)

Benchmarking all-reduce, all-gather and friends across interconnect topologies

Duration	Core technologies
7 weeks	NCCL, CUDA, MPI / torch.distributed, nccl-tests, Python

1. Overview

Distributed training speed is often limited by GPU-to-GPU communication. This project builds a benchmark suite for NCCL collectives across message sizes, GPU counts and interconnects (NVLink, PCIe), and turns the results into guidance.

Research question

Which collective algorithms, protocols and settings perform best for each topology and message size?

Expected outcome

A benchmark report identifying optimal collective strategies per topology.

Prerequisites and hardware

Skills: Python, basic distributed training concepts, Linux command line.

Hardware: Requires 2 or more GPUs in one node (4-8 ideal). If you lack them, rent a multi-GPU cloud instance for a few hours and plan the runs in advance. Multi-node is optional.

2. Architecture / pipeline

Topology discovery (nvidia-smi topo -m) -> launcher (torchrun or mpirun) -> collective benchmark loop (message-size sweep) -> metrics (latency, algbw, busbw) -> environment-variable experiments -> report with per-topology recommendations.

3. What to build

- Validate setup with NVIDIA's nccl-tests (all_reduce_perf, all_gather_perf) as the reference.
- Own harness using torch.distributed with the NCCL backend: all-reduce, all-gather, reduce-scatter, broadcast, all-to-all.
- Message sizes from 1 KB to 1 GB, warmup and many iterations, CUDA event timing.
- Compute algorithm bandwidth and bus bandwidth using the standard nccl-tests formulas.
- Env-var experiments: NCCL_ALGO (ring/tree), NCCL_PROTO (LL, LL128, Simple), NCCL_P2P_DISABLE, NCCL_DEBUG=INFO for logs.
- Compute-communication overlap test (collective on a separate stream while a matmul runs); DDP bucket-size study.
- Topology report: map results to the interconnect layout.

4. Baselines to compare

nccl-tests reference numbers | Your harness (should match nccl-tests) | Default NCCL settings vs tuned settings | Theoretical link bandwidth

5. Experiments to run

Message size sweep | GPU count scaling (2, 4, 8) | NVLink vs PCIe paths (P2P on/off) | Ring vs tree algorithm | Protocol selection | dtype (FP32 / FP16 / BF16) | Compute-communication overlap | DDP bucket_cap_mb sweep

6. Evaluation metrics

Latency for small messages | Algorithm and bus bandwidth | Percent of theoretical bandwidth | Scaling efficiency | Overlap efficiency

7. Suggested repository structure

```
nccl_bench/
  bench/  launchers/  topo/
  experiments/  results/  plots/
  docker/  tests/  docs/
```

8. Week-by-week plan (7 weeks)

Week	Deliverable
1	Hardware access plan, topology discovery, run nccl-tests.
2	Own harness for all-reduce; validate vs nccl-tests.
3	Add all-gather, reduce-scatter, broadcast, all-to-all.
4	Env-var experiments (algo, proto, P2P).
5	GPU-count scaling and topology analysis.
6	Overlap and DDP bucket experiments.
7	Report with recommendations, demo.

9. Definition of done

- Harness validated against nccl-tests
- All listed collectives benchmarked
- Per-topology recommendation table
- Raw data and scripts included

10. Final deliverables

- Benchmark suite and launchers
- Result datasets and plots
- Topology and tuning guidance report
- Reproducible Docker/run scripts

11. Critical engineering rules

- Hardware access is the main risk; secure GPUs early.
- Record driver, CUDA, NCCL versions and exact topology with every result.
- Use bus bandwidth for cross-collective comparison.
- Warm up and repeat; first iterations include setup cost.

Project 8: GPU Kernel Fusion Compiler Pass for a Small DL Model

Fusing elementwise operations into single kernels with torch.fx and Triton

Duration	Core technologies
9 weeks	PyTorch 2.x, torch.fx, TorchInductor, Triton

1. Overview

Unfused elementwise ops each read and write GPU memory, wasting bandwidth and launches. This project implements a graph pass that finds chains of elementwise ops, generates one fused Triton kernel for each, and measures the effect on a small model.

Research question

How much do kernel count, memory traffic and latency improve from your fusion pass, and how does it compare with TorchInductor's own fusion?

Expected outcome

A fusion pass that reduces kernel-launch count and improves end-to-end latency.

Prerequisites and hardware

Skills: PyTorch, Python metaprogramming basics, some Triton.

Hardware: Any NVIDIA GPU supported by PyTorch 2.x and Triton.

2. Architecture / pipeline

Model -> capture FX graph (torch.compile custom backend or torch.fx tracing) -> pattern detection of elementwise chains -> fused Triton kernel codegen -> graph rewrite -> run -> correctness and performance comparison.

3. What to build

- Target model: MLP with bias + GELU + dropout + residual, or a small transformer / ResNet block.
- Capture graph with a custom torch.compile backend (receives an FX GraphModule) and inspect nodes.
- Fusion analysis: find maximal connected chains of pointwise ops with compatible shapes and no side-uses.
- Codegen: generate Triton kernel source (or use templates) for each chain, handle broadcasting and dtypes.
- Graph rewrite: replace the chain with a call to the fused kernel; keep fallback to eager for unsupported ops.
- Optional: fusing into backward for training.
- Benchmark and profile with torch.profiler (kernel count) and memory-traffic estimates.
- Note: internal TorchInductor extension points change between versions; verify the API for your installed version.

4. Baselines to compare

Eager PyTorch | torch.compile default (Inductor already fuses many ops) | Your FX fusion pass

5. Experiments to run

Chain length | Tensor size | dtype | Inference vs training | Dynamic shapes | Fused vs unfused memory traffic | Compile-time overhead

6. Evaluation metrics

Kernel launch count | Estimated bytes moved | End-to-end latency | Speedup vs eager and vs Inductor | Max numerical error | Compile time

7. Suggested repository structure

```
fusion_pass/
  backend/  patterns/  codegen/
  models/  bench/  tests/  docs/
```

8. Week-by-week plan (9 weeks)

Week	Deliverable
1	Learn torch.fx and torch.compile backends; choose model; baseline numbers.
2	Graph capture and inspection tooling.
3	Pattern detection of elementwise chains.
4	Hand-written fused Triton kernel for one pattern.
5	Codegen for general chains.
6	Graph rewrite and correctness tests.
7	Benchmarks vs eager and Inductor.
8	Training/backward or dynamic shapes extension.
9	Report, demo.

9. Definition of done

- Pass fuses at least one realistic pattern
- Outputs match eager within tolerance
- Kernel count reduction shown
- Comparison with Inductor

10. Final deliverables

- Compiler pass and backend
- Generated-kernel examples
- Benchmark report
- Demo

11. Critical engineering rules

- Inductor already fuses many ops; compare against it honestly.
- Only fuse when semantics and shapes allow; test edge cases.
- Use pinned PyTorch and Triton versions; internal APIs change.

Project 9: Mixed-Precision CUDA Training Kernel Benchmarks

Throughput versus convergence trade-offs of FP32, TF32, FP16, BF16 and FP8

Duration	Core technologies
8 weeks	CUDA, PyTorch AMP, Transformer Engine, Nsight Systems

1. Overview

Lower precision increases throughput and reduces memory but can hurt stability. This project benchmarks precision modes on your target GPUs and models, measuring both speed and convergence, and produces a decision guide.

Research question

For each target GPU and model, which precision gives the best throughput without hurting convergence?

Expected outcome

A benchmark report guiding precision choice for target hardware and model combinations.

Prerequisites and hardware

Skills: PyTorch training loops, basic numerics (floating-point formats).

Hardware: FP16 needs Volta+; BF16 and TF32 need Ampere+; FP8 via Transformer Engine needs Hopper or Ada (e.g. H100, L4, RTX 40). A T4 cannot run FP8 or BF16 efficiently.

2. Architecture / pipeline

Models (small GPT, ResNet, ViT) -> precision configurations -> fixed-token/epoch training runs -> throughput, memory and loss logging -> stability analysis -> decision table.

3. What to build

- Models: small GPT-style transformer and a CNN or ViT; fixed seeds and datasets.
- Precision configs: FP32, TF32, FP16 with GradScaler, BF16 autocast, FP8 with Transformer Engine (if hardware allows).
- Training harness logging step time, tokens or images per second, peak memory, loss, gradient norm, overflow/NaN events.
- Kernel-level microbenchmarks: GEMM throughput per precision.
- Numerics study: which ops stay in FP32 (norms, softmax, loss), loss-scale behavior.
- Convergence criterion: same token or epoch budget, compare final validation loss or accuracy over at least 3 seeds.
- Decision guide: table of recommended precision per GPU, model size and sequence length.

4. Baselines to compare

FP32 (TF32 off) | TF32 | FP16 AMP | BF16 AMP | FP8 (if supported)

5. Experiments to run

Model size | Batch size and sequence length | Learning-rate sensitivity per precision | 3+ seeds for variance | Layers kept in FP32 | GEMM-only vs full-model speedup | Different GPUs

6. Evaluation metrics

Tokens/s or images/s | Step time | Peak memory | Model FLOPs utilization | Final validation loss / accuracy | Loss spikes, NaNs, overflow counts

7. Suggested repository structure

```
mixed_precision_bench/
  models/  precision/  train/
  microbench/  results/  plots/
  configs/  tests/  docs/
```

8. Week-by-week plan (8 weeks)

Week	Deliverable
1	Hardware inventory, models, data, FP32 baseline.
2	AMP FP16 and BF16 runs; harness metrics.
3	GEMM microbenchmarks per precision.
4	FP8 / Transformer Engine (if available) or extended TF32/AMP study.
5	Convergence runs with multiple seeds.
6	Stability analysis: LR sensitivity, overflows, FP32 islands.
7	Cross-GPU or cross-model runs.
8	Decision guide, report, demo.

9. Definition of done

- Throughput and convergence for each precision
- 3+ seeds with variance
- Decision table per GPU/model
- Documented hardware limitations

10. Final deliverables

- Benchmark harness and configs
- Result tables and plots
- Precision decision guide
- Technical report

11. Critical engineering rules

- Compare convergence at equal data budgets, not equal steps only.
- Faster does not mean acceptable; report accuracy and stability.
- Do not claim FP8 results without supporting hardware.
- Run multiple seeds; single runs are misleading.

Project 10: Custom CUDA Kernel for Sparse Matrix Operations in ML Pipelines

Sparse matmul tuned to your model's sparsity pattern, compared with cuSPARSE and dense

Duration	Core technologies
9 weeks	CUDA C++, cuSPARSE baseline, cuSPARSELt (2:4), PyTorch extension

1. Overview

Generic sparse libraries are not tuned to the structured patterns real models have. This project characterizes your model's sparsity, builds a sparse-times-dense kernel (SpMM) tuned to it, and compares against cuSPARSE and, critically, against dense matmul.

Research question

At which sparsity levels and structures does a custom sparse kernel beat cuSPARSE and dense cuBLAS?

Expected outcome

A sparse kernel that outperforms cuSPARSE on the target sparsity pattern.

Prerequisites and hardware

Skills: CUDA kernels, sparse formats (CSR), PyTorch basics.

Hardware: NVIDIA GPU; 2:4 structured sparsity with sparse Tensor Cores needs Ampere or newer.

2. Architecture / pipeline

Sparsity analysis -> format choice (CSR, BSR, ELL, blocked, 2:4) -> kernel design (row-split, block-based, load balancing) -> correctness vs dense masked -> benchmark vs cuSPARSE and cuBLAS -> PyTorch extension -> end-to-end layer test.

3. What to build

- Sparsity characterization: sparsity level, block structure, row-length distribution of your target weights or masks.
- Format converters: dense to CSR / BSR / blocked formats.
- Baselines: dense cuBLAS matmul, cuSPARSE SpMM (torch.sparse.mm with CSR), optional Triton block-sparse.
- Kernel v1: row-per-warp CSR SpMM; v2: block-sparse with shared-memory tiles; v3: load-balanced variant for skewed rows.
- Optional: 2:4 structured sparsity with cuSPARSELt as a hardware-supported baseline.
- Correctness: compare with dense matmul using the masked matrix.
- PyTorch extension and end-to-end test inside a linear layer or small model (accuracy retention if you prune).

4. Baselines to compare

Dense cuBLAS | cuSPARSE CSR SpMM | Naive custom sparse kernel | Optimized custom kernel

5. Experiments to run

Sparsity level sweep (50% to 99%) | Block size | Format comparison | Dense-matrix width (N) | Row-length skew and load balancing | dtype | End-to-end layer latency and accuracy

6. Evaluation metrics

Latency | Speedup vs cuSPARSE and vs dense | Effective GFLOPS | Memory bandwidth | Accuracy retention if pruned | Break-even sparsity vs dense

7. Suggested repository structure

```
sparse_kernels/
  formats/  kernels/  python/
  bench/  patterns/  tests/
  results/ docs/
```

8. Week-by-week plan (9 weeks)

Week	Deliverable
1	Analyze target sparsity; dense and cuSPARSE baselines.
2	Format converters and correctness harness.
3	CSR row-per-warp SpMM kernel.
4	Optimization: vectorized loads, shared-memory reuse.
5	Block-sparse kernel.
6	Load-balanced variant for skewed rows.
7	Sweeps over sparsity, block size, N.
8	PyTorch extension and end-to-end test.
9	Break-even analysis, report, demo.

9. Definition of done

- Kernel passes correctness tests
- Beats cuSPARSE on target pattern, or documented why not
- Compared against dense with break-even point
- End-to-end layer demo

10. Final deliverables

- Sparse kernels and converters
- Benchmark data and plots
- Break-even analysis
- Technical report and demo

11. Critical engineering rules

- Always compare with dense cuBLAS; dense often wins until sparsity is very high.
- Your pattern must match your real model; random sparsity results do not transfer.
- Include format-conversion cost when relevant.
- Verify correctness against masked dense matmul.